page handlers in the Razor Page.

Adding the `[Authorize]` attribute fulfills your first requirement, but unfortunately, with the techniques you've seen so far, you have no way to fulfill the second. You could apply a policy that either permits or denies a user the ability to edit all recipes, but there's currently no easy way to restrict this so that a user can only edit their own recipes.

To find out who created the `Recipe`, you must first load it from the database. Only then can you attempt to authorize the user, taking the specific recipe (resource) into account. The following listing shows a partially implemented page handler for how this might look, where authorization occurs partway through the method, after the `Recipe` object has been loaded.

Listing 24.13 The Edit.cshtml page must load the `Recipe`

```
public IActionResult OnGet(int id)                ❶
{
    var recipe = _service.GetRecipe(id);          ❷
    var createdById = recipe.CreatedById;         ❷
    // Authorize user based on createdById        ❸
    if(isAuthorized)                              ❹
    {                                             ❹
        return View(recipe);                      ❹
    }                                             ❹
}
```

❶ The id of the recipe to edit is provided by model binding.

❷ You must load the Recipe from the database before you know who created it.

❸ You must authorize the current user to verify that they're allowed to edit this specific Recipe.

❹ The action method can continue only if the user was authorized.

You need access to the resource (in this case, the `Recipe` entity) to perform the authorization, so the declarative `[Authorize]` attribute